

PassportSIM — Supabase Integration, Accounts & Deployment

What was connected, what was built, the decisions behind each tradeoff, and what still needs a real credential before this is production-ready.

PREPARED FOR

Yvette's supervisor

PREPARED BY

**Claude, with Yvette
Christine**

DATE

24 August 2026

REPOSITORY

dharmi2002/swift-e-
connect-

● Supabase database — live

● Account login — unverified emails

● eSIM delivery — simulated

● Vercel — not yet deployed

ON THIS PAGE

- | | |
|------------------------------------|------------------------------------|
| 01 Supabase connected & verified | 06 GitHub repository change |
| 02 Account creation, built | 07 Vercel deployment status |
| 03 How login works today & its gap | 08 Open items at a glance |
| 04 Brevo disclaimer | 09 Appendix: environment variables |
| 05 Checkout runs in simulated mode | |

01 Supabase connected & verified

The storefront is now wired to a live Supabase project, and every table the app depends on was confirmed to actually exist — not assumed from the migration files.

The repository’s configuration originally pointed at a Supabase project reference that was never actually populated with real credentials. Once the real project (1vaadadxbgyoazchwekc) was provided, we corrected that reference and checked it directly: it had **zero tables**. The three migration files in the repo had never been run against it.

DECISION

Rather than trust that migrations had run, we queried the live database directly (via its REST layer) both before and after applying them. This caught the empty-database issue immediately instead of it surfacing later as a broken storefront.

All three migrations are now applied and confirmed live:

MIGRATION	WHAT IT CREATES	VERIFIED
20260819175906_....sql	packages (16 seeded eSIM data plans — Kenya, Nigeria, South Africa, Ghana, Egypt, Morocco, UAE, UK, USA, plus regional passes) and orders	• Live
20260819180100_backend_tables.sql	webhook_logs , system_settings (seeded), extra order columns for the eSIM provider integration	• Live
20260824103000_profiles_and_order_accounts.sql (new)	profiles table, auto-created on signup; links orders to the	• Live

account that
placed them

02 Account creation, built

Sign-up and login pages were added on top of the existing plan-selection storefront, without disturbing how guest checkout already worked.

New **Sign up** and **Sign in** pages let a customer create an account with an email and password. Once logged in, the header shows their email and a sign-out control, and the checkout form now pre-fills their email automatically.

Under the hood, a new `profiles` record is created automatically the moment someone signs up, and any order they place afterward is linked to their account — while an order placed without logging in still works exactly as before. Guest checkout was a deliberate part of the original storefront design, and this work preserves it rather than forcing account creation.

03 How login works today, and its gap

Authentication is handled by Supabase Auth, the same provider hosting the database — the app never stores or sees raw passwords. That part is solid. What's missing is proof that a signup's email address is real. ▶

Right now, **email confirmation is switched off**. Anyone can create an account with any email address — real or made up — and get full access immediately, with no verification step at all. This was a deliberate, sequenced decision, not an oversight:

- 1 We initially kept email confirmation **on**, planning to use a real inbox during a demo recording so the walkthrough showed the genuine flow.
- 2 That triggered Supabase's default email sender rate limit — the free sender Supabase provides out of the box is intentionally capped very low and explicitly meant for testing, not real signups.
- 3 To keep testing moving, we switched email confirmation **off** entirely. This unblocks development today but is not where the project should stay once it has real

customers.

The fix is to give Supabase a real email sender with a higher limit, then turn confirmation back on:

- 1 Get Brevo SMTP credentials (see the disclaimer below) — Brevo’s free tier allows 300 emails/day, far above Supabase’s default.
- 2 Add those credentials in Supabase under **Authentication** → **SMTP Settings**.
- 3 Verify the sending domain in Brevo (SPF/DKIM records) so confirmation emails don’t land in spam.
- 4 Turn **“Confirm email”** back on under **Authentication** → **Sign In / Providers** → **Email**.
- 5 Run one real signup end-to-end to confirm the email arrives and the link works before treating it as done.

04 Brevo disclaimer

NOT YET CONNECTED

Brevo is this project’s email provider, already referenced in the codebase for one specific job: emailing a customer their eSIM QR code after a real purchase. That connection uses a placeholder key today — no real Brevo account is linked yet, for order emails or for login emails.

Two separate things depend on a real Brevo account: **order confirmation emails** won’t send to real customers until the API key is real, and **email verification** for new accounts can’t safely be re-enabled until Brevo’s SMTP credentials are configured (otherwise Supabase’s rate limit problem returns immediately). Nothing breaks by leaving this for later — it simply gates those two features.

05 Checkout runs in simulated mode

Provisioning a real eSIM requires a paid wholesale account with eSIMAccess, which isn’t connected yet either — so checkout was given a demo fallback.

When a customer completes checkout today, the app skips the real eSIMAccess call and instead completes the order immediately with realistic-looking mock delivery details: a QR code, an activation code, and an ICCID. This lets the full plan-selection → checkout → “eSIM ready” flow be demonstrated and tested end-to-end without a live paid account.

DECISION

The fallback only activates while the eSIMAccess key is a placeholder. The moment a real key is added, the app automatically switches to real provisioning — no code change required to turn simulated mode off.

06 GitHub repository change

The repository originally pointed to a GitHub location (`01i-thyn-code/swift-e-connect`) that turned out not to be accessible — not owned by any credential available, and confirmed as not something Yvette has access to. Work is now pushed to `github.com/dharmi2002/swift-e-connect` - instead, which is confirmed up to date with everything described in this memo.

07 Vercel deployment status

The build system was tested with a simulated Vercel environment and confirmed to auto-detect the target and produce correct deployment output — **no code changes are needed** to deploy this app to Vercel. Actually creating the Vercel project and going live is the one step still pending, and needs manual steps in the Vercel dashboard (importing the GitHub repo and setting environment variables — listed in the appendix).

08 Open items at a glance

ITEM	CURRENT STATE	RISK IF LEFT AS-IS	ACTION NEEDED
Email verification	• Off	Anyone can sign up with a fake or unowned email	Configure Brevo SMTP in Supabase, re-enable confirmation
eSIM provisioning	• Simulated	No real customer receives an actual eSIM	Obtain a real eSIMAccess wholesale API key
Order confirmation emails	• Not sending	Customers get no purchase confirmation email	Obtain a real Brevo API key
Vercel deployment	• Not live	App is reachable only on a local machine	Import repo in Vercel, set env vars, deploy
GitHub repository owner	• dharmi2002	May not be the intended long-term home	Confirm, or migrate to a permanent org account

09 Appendix — environment variables for Vercel

These need to be set in the Vercel project's settings before the first deploy. Real values live in the local `.env` file (not committed to GitHub).

VARIABLE	STATUS	NOTES
SUPABASE_URL	• Real	Live project URL
VITE_SUPABASE_URL	• Real	Same value, client-side build
SUPABASE_PUBLISHABLE_KEY	• Real	Public anon key
VITE_SUPABASE_PUBLISHABLE_KEY	• Real	Same anon key, client-side build
SUPABASE_SERVICE_ROLE_KEY	• Real	Server-only, bypasses row security — keep secret
ESIM_ACCESS_API_KEY	• Placeholder	Leave as-is to keep simulated checkout

VARIABLE	STATUS	NOTES
ESIM_ACCESS_BASE_URL	• Set	eSIMAccess wholesale API endpoint
BREVO_API_KEY	• Placeholder	Needed for real order emails
FROM_EMAIL / FROM_NAME	• Placeholder	Sender identity for order emails
CRON_SECRET	• Placeholder	Any random string, protects admin endpoints